# SAILOR: Systematic Tool Support for Systematic Literature Reviews

Jan Keim
Karlsruhe Institute of Technology (KIT)
Karlsruhe, Germany
jan.keim@kit.edu

***Abstract*—Systematic literature reviews are a common research method in software engineering. They follow a protocol so that other researchers can trace and repeat each decision. However, the review data is hard to keep traceable. Reviewers record screening decisions and extracted values in spreadsheets, shared documents, or hosted services. There, the link between an extracted value and its evidence in the paper can get lost. Also, it is hard to reconstruct who decided what and when. To tackle this, we present SAILOR, an open-source desktop tool that supports screening, data extraction, and consolidation of a review in one project. The project is a set of plain JavaScript Object Notation (JSON) files that a schema describes and that git versions. Several reviewers work independently in their own seats, and a consolidator resolves their disagreements with agreement statistics. When reviewers synchronize through git, the tool merges concurrent changes field by field. Optional large language model support proposes answers with quoted evidence. A human reviews every proposal. We describe the tool from the reviewer's view and explain its main architecture decisions, including the move from Electron to Tauri and the support for large projects. We also report how we keep the tool itself traceable. We have not yet evaluated the tool with users. We documented 510 requirements and linked them to code, tests, and documentation. Of these requirements, 95% link to implementing source code and 94% link to at least one automated test. The test suites hold 3709 unit tests, 56 integration tests, and 135 Rust tests. The source code of SAILOR is available on GitHub under the GPL-3.0 license. Its Zenodo concept digital object identifier (DOI) points to the latest archived version. An online demo runs SAILOR in the browser without an installation.**

***Index Terms*—Systematic Literature Review, Tool Support, Data Extraction, Requirements Traceability, Large Language Models**

## I. INTRODUCTION

systematic literature reviews (SLRs) are an established research method in software engineering (SE). Kitchenham and Charters [1] adapted the method from medicine, and later guidelines added systematic mapping studies and snowballing [2], [3]. The core promise of an SLR is that other researchers can follow each step. Therefore, reviewers define a protocol up front, select papers with explicit criteria, and extract data with a fixed form. Reporting guidelines such as Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) [4] then ask authors to disclose how many papers they found, screened, and excluded, and why.

However, the data that an SLR produces is hard to keep traceable. Reviewers often record screening decisions and extracted values in spreadsheets, shared documents, or hosted services. In such a setup, the link between an extracted value and the passage in the paper that supports it is often only in the reviewer's head. Also, several reviewers can extract the same papers independently, and someone has to compare and consolidate their answers. With a shared spreadsheet, it is hard to see who changed which cell and why. Recently, large language models (LLMs) have been proposed for parts of screening and extraction. Yet studies find that their output still needs human control [5], [6].

Several tools support parts of the process. Mapping studies and evaluations cover many of them [7]–[9]. Many of these tools are hosted web applications that keep the review data on a server, for example, Covidence [10], Rayyan [11], and CADIMA [12]. Other tools focus on one activity, such as screening with active learning in ASReview [13]. To the best of our knowledge, no tool combines local, plain-file review data with a version history, schema-driven extraction with evidence links, independent reviewers with a consolidation step, and LLM proposals that a human must accept.

In this paper, we present SAILOR, an open-source desktop tool that supports SLRs in a systematic way. The name SaiLoR is built from S, ai, L, o, and R, where the letter o only connects the parts. The capital letters S, L, and R stand for SLR, the research method that the tool is developed for. The lowercase letters ai stand for artificial intelligence (AI), which the tool already includes as LLM assistance in preview and which we want to extend. The main idea is to treat review data like source code. A review is a project of plain JSON files. A schema describes what reviewers extract, and the tool validates every answer against it. Each reviewer writes into their own files, and git records every change. When reviewers pull each other's work, the tool merges their files field by field. Optional LLM assistance proposes answers with quotes from the paper. The reviewer decides which proposal to apply, and the project records that an LLM was involved.

We use the example project that ships with SAILOR as a running example. The project is a small review on "Code Search and Program Repair". A screening project holds eight candidate papers. Five of them are included and carried into an extraction project with a nested schema. Figure 1 shows this extraction project in the main view of SAILOR.

In summary, we make the following contributions:

- We present SAILOR, a desktop tool that supports screening, data extraction, and consolidation of an SLR with

several reviewers.
- We describe the main architecture decisions behind SAILOR and their reasons, including local-first storage, git as collaboration layer, the move from Electron to Tauri, the support for large projects, and the control of LLM assistance.
- We report how we keep the tool traceable with 510 requirements that we linked to source code, tests, and documentation, and how we tested the tests with mutation testing.
- We make the source code of SAILOR available on GitHub under the GPL-3.0 license, with a Zenodo concept DOI [14] for all versions.

The remainder of the paper is structured as follows. In Section II, we discuss related work. In Section III, we explain which SLR activities SAILOR supports and which design principles guide it. In Section IV, we walk through the tool from the reviewer's view. In Section V, we describe the main architecture decisions. In Section VI, we present the requirements and their trace links. We discuss limitations in Section VII and conclude in Section VIII.

## II. Related Work

In this section, we give an overview of related work on tool support for SLRs. We first look at the methodology and at studies about SLR tools. Then, we discuss existing tools and, lastly, the use of LLMs in SLRs.

### A. Methodology and Studies on Tool Support

The SLR method in SE goes back to the guidelines by Kitchenham and Charters [1] and the book by Kitchenham et al. [15]. Petersen et al. [2] describe systematic mapping studies, and Wohlin [3] describes snowballing as a search strategy. PRISMA 2020 [4] defines what a systematic review reports, including a flow diagram of found, screened, and included studies. These guidelines define the activities that SAILOR supports. SAILOR does not add new methodology. Instead, it records the data that these guidelines ask for.

Several studies look at tools for SLRs. Marshall and Brereton [7] map the tools that SE researchers proposed, and Marshall et al. [8] compare the features of SLR tools in SE. Hassler et al. [16] collected barriers to the SLR process in a community workshop. Al-Zubidy and Carver [17] derived requirements for tools that support the search activity. Felizardo and Carver [18] give an overview of how SLR activities can be automated, and van Dinter et al. [19] review automation approaches in an SLR of their own. Most of the studies that van Dinter et al. found automate the selection of primary studies. Outside SE, Harrison et al. [9] evaluate tools for title and abstract screening in healthcare. These studies show that tool support exists for many activities. Our work differs in its goal. We do not survey or compare tools but describe the design of one tool and how we keep it traceable.

### B. Tools for Systematic Literature Reviews

A large group of tools are hosted web applications. Covidence [10] is a web platform that covers screening, full-text review, risk of bias, and data extraction. Rayyan [11] is a web and mobile app that supports screening with semi-automated suggestions. CADIMA [12] is an open-access online tool by the Julius Kühn-Institut for the conduct and reporting of systematic reviews and maps. Parsifal [20] is an online tool that supports researchers in planning, conducting, and reporting SLRs. These tools are web applications, so the review data is typically held on a server, and collaboration happens in the shared online project. SAILOR takes the opposite route. The review data stays in local files, and reviewers exchange it through git, a hosting service of their choice, or not at all. Therefore, reviewers can inspect, archive, and publish the raw data with standard tools.

Some tools come from SE research. StArt [21], [22] supports the whole systematic review process, and Hernandes et al. [21] evaluated it with goal question metric (GQM) and technology acceptance model (TAM). SLR-Tool [23] is a free tool that follows the process by Kitchenham and adds text mining and export to BibTeX and RIS. Both tools target the same community as SAILOR. To our knowledge, the publications on these tools do not describe independent extraction by several reviewers with a consolidation step, or collaboration through version control.

ASReview [13] is an open-source framework that uses active learning to rank the records that a reviewer should screen next. Like SAILOR, ASReview can run locally. However, ASReview focuses on screening, while SAILOR focuses on extraction and consolidation and offers screening as a preceding step. SAILOR does not rank records with active learning. The two tools can complement each other, since SAILOR imports common reference formats such as BibTeX and RIS.

### C. Large Language Models in Systematic Literature Reviews

Recently, many studies evaluate LLMs for single SLR activities. For selection, Felizardo et al. [5] evaluate the accuracy of ChatGPT on SLRs in SE, and Syriani et al. [24] use ChatGPT to screen articles for systematic reviews. Felizardo et al. report 75.3% and 86.1% accuracy in the selection activity of two SLRs. They advise against fully outsourcing the selection and see ChatGPT as a support tool. For extraction, Felizardo et al. [25] and Gartlehner et al. [26] report proof-of-concept studies. Khraisha et al. [6] test GPT-4 for screening and extraction across languages. After correcting for chance agreement and dataset imbalance, they find only moderate performance for data extraction. Other work uses ChatGPT to formulate Boolean search queries [27] or across several stages of a review [28]. These studies focus on model performance. They say less about how a tool should present model output, so that the reviewer stays in control and the use of the model stays visible. SAILOR addresses this second question. Every proposal is a suggestion that a reviewer must apply, the model can act as an additional reviewer whose answers go through

TABLE I
SLR PHASES AND ACTIVITIES [1] AND THEIR SUPPORT IN SAILOR

| Activity | Support in SAILOR |
|---|---|
| *Planning* | |
| Protocol | Research questions, search strings, databases, search date, and notes in the project |
| Extraction form | Schema of nested fields with types, options, and visibility conditions |
| *Conducting* | |
| Search | Not supported. Import of BibTeX, RIS, CSL-JSON, and portable document format (PDF) folders |
| Duplicates | Classification into new, certain, and probable duplicates with a human decision |
| Screening | Include, exclude with reason, or leave undecided per reviewer |
| Extraction | Schema-driven form next to the PDF with highlights and evidence links |
| Consolidation | Comparison of reviewers with agreement statistics and adoption of answers |
| *Reporting* | |
| Selection counts | Screening summary with counts per exclusion reason |
| Exports | Disagreements as text, marks as standard PDF annotations, knowledge graph |
| LLM use | Durable record of each applied LLM proposal per paper |
| Synthesis | Not supported. Extracted data is plain JSON, queries on the knowledge graph |

the same consolidation, and the project records each use of the model.

Overall, most existing tools keep the review data on a server or focus on a single activity. To the best of our knowledge, no tool yet combines local, versioned review data, schema-driven extraction with evidence links, consolidation of independent reviewers, and controlled LLM assistance.

## III. SUPPORTING SLRS SYSTEMATICALLY

In this section, we explain which SLR activities SAILOR supports and which principles guide its design. We then introduce the running example in more detail.

### A. Covered Activities

Kitchenham and Charters [1] split an SLR into three phases: planning, conducting, and reporting. Table I maps these phases to the support in SAILOR. The focus lies on the conducting phase. In this phase, several reviewers handle the same papers, and most review data arises. SAILOR does not run the database search itself. Reviewers search in digital libraries as before and import the exported references. SAILOR also does not synthesize the results. Synthesis depends on the research questions. The extracted data is plain JSON that reviewers can analyze with their own scripts.

### B. Design Principles

We built SAILOR on five principles. Each principle follows from the goal that other researchers should be able to trace the review data.

*a) Local, plain files:* A project is a `project.json` file with the metadata, the schema, and the papers. A sibling `annotations` folder holds one file per paper and reviewer. Kleppmann et al. [29] argue that users should own their data and keep working without a server. For an SLR, local files have a further benefit. Reviewers can publish the raw review data with the paper, and readers do not need SAILOR to read it.

*b) The schema drives extraction:* Reviewers define the extraction form as a schema. A schema consists of nodes. A node is either a field of type string, number, boolean, or year, or a group of further nodes. A node can be required, can have a list of options, and can repeat between a minimum and maximum count. A visibility condition hides a node until another field has a certain answer. SAILOR validates every answer against the schema, and it keeps hidden answers instead of deleting them. When reviewers rename a node, SAILOR carries the existing answers over.

*c) Independent reviewers and explicit consolidation:* Each reviewer works in their own *seat*. A project has between two and ten reviewer seats and one additional consolidation seat. Reviewers do not see each other's answers while they extract. The *consolidator* then compares the seats and decides on the final answer. In this way, the project keeps the individual answers and the consolidated result side by side.

*d) A recorded history:* SAILOR uses git to record and share every change. Ram [30] argues that git makes scientific work more transparent and reproducible. We follow this argument for review data. A commit shows who changed which answer and when, and a merge combines the work of reviewers who worked in parallel.

*e) The human decides:* SAILOR proposes and the reviewer decides. This principle applies to duplicates, to screening, and to LLM assistance. For example, SAILOR never merges a probable duplicate on its own, and it never writes an LLM answer into the project before the reviewer applies it. LLM support is off by default, and each project must enable it.

### C. Running Example

The running example consists of two sample projects in the repository. The screening project contains eight candidate papers on code search and program repair. It defines eight exclusion reasons, for example, "Wrong topic", "Not peer-reviewed", and "Duplicate". In the example, the screening result includes five papers and excludes three. SAILOR then creates the extraction project "Example SLR: Code Search and Program Repair" from the screening result. The extraction project carries over the five included papers, and it records that it was created from a screening project. Its schema asks whether a paper is relevant, which study type it reports, and in which year it appeared. A repeatable group *Findings* holds a claim, the evidence for the claim with metric and evaluation type, and a confidence value. A second group *Threats to Validity* repeats at most three times and records the kind of each threat and whether the authors addressed it. Both projects

have two reviewer seats and allow LLM assistance. We use this example in the walkthrough in the next section.

## IV. SAILOR FROM THE REVIEWER'S VIEW

In this section, we walk through SAILOR along the activities of an SLR. We use the running example from Section III-C. Some figures show the online demo of SAILOR instead, which runs in the browser with its own project of seven papers. The demo can show the annotation, consolidation, and knowledge graph views, and it opens the project editor on a new project. It does not save, and it offers neither the git view nor the LLM assistance.

### A. Planning: Protocol and Schema

Reviewers start in the project editor. There, they set the project title, the number of reviewer seats, and whether the project is a screening project. An optional review protocol records the research questions, search strings, databases, search date, and further notes. The main part of the editor is the schema, as shown in Figure 2. Each row defines one node with its name, type, minimum and maximum count, options, description, and visibility condition. In the figure, the field *Exclusion Reason* is only visible if the field *Relevant* has the answer "No". Reviewers can reorder and nest rows by drag and drop.

### B. Conducting: Import and Screening

Reviewers import references from BibTeX, RIS, or CSL-JSON files, and they can import a folder of PDFs including its subfolders. During import, SAILOR sorts each record into one of three classes: new, certain duplicate, or probable duplicate. A certain duplicate has the same normalized DOI or title as a paper in the project. A probable duplicate has a fuzzy title similarity of at least $0.90$, or the same base title and an author similarity of at least $0.50$. The reviewer must decide on each probable duplicate. If the reviewer merges two records, SAILOR only fills fields that are empty in the existing paper.

A project can also name a PDF folder inside the project folder. SAILOR scans this folder when the project opens, when the window gets the focus, and when the reviewer asks for a rescan. It compares the files by their SHA-256 hash and not by their names, so a renamed file is not offered again. For each new file, SAILOR proposes the title, the authors, the year, and the DOI. The proposal comes from the properties and the first page of the PDF, optionally from Crossref, and otherwise from the file name. SAILOR does not use an LLM for this proposal.

In a screening project, each reviewer decides per paper whether to include it, to exclude it with a reason, or to leave it undecided. Figure 3 shows the screening view of the running example. The view shows the abstract instead of the PDF, because screening decisions rest on titles and abstracts. If the abstract was extracted from the PDF and not imported, SAILOR shows a warning, because the extraction can be wrong. Reviewers can screen with the keyboard: `I` includes, `E` excludes, `U` clears the decision, and the number keys exclude with the matching reason. After a decision, SAILOR moves on to the next paper. A progress line shows how many papers the reviewer screened, included, and excluded. The screening summary lists these counts with one row per exclusion reason, which is the input for a PRISMA flow diagram [4]. Validation reports excluded papers without a reason.

When screening is done, reviewers create the extraction project from the screening project. SAILOR carries over all included papers, never carries over excluded papers, and lets the reviewer choose whether to carry over undecided papers. The new project records where it came from and how many papers were included, excluded, and carried over.

### C. Conducting: Data Extraction

Figure 1 shows the main view during extraction in the online demo. The left pane lists the papers, the middle pane shows the PDF, and the right pane shows the form that SAILOR generates from the schema. Repeatable groups such as *Findings* get buttons to add and remove entries. A dot next to each paper shows how complete the paper is for the current seat. Reviewers mark a paper as finished explicitly. Thus, "finished" is a decision of the reviewer and not just the absence of empty fields.

Reviewers can copy selected text from the PDF into a field. They can also highlight passages in five colors and add notes. Each reviewer seat has its own marks. A reviewer can link a mark to one or more fields as evidence. The link popover of a field lists the linked marks, and a click on a mark jumps to the passage in the PDF. In this way, the project stores where in the paper an answer comes from. Reviewers can export their marks as standard PDF annotations into a new PDF file.

Imported metadata can be wrong, for example, a title with broken characters or a missing DOI. If the project enables metadata correction, reviewers can correct the title, the authors, the year, and the DOI in the annotation view. SAILOR stores each correction in the annotation file of the reviewer's seat and not in the project file. The consolidator compares the corrections of all seats in a metadata dialog and then updates the project metadata.

### D. Conducting: Consolidation

In a project with several reviewers, every reviewer must pick a seat before editing. A reviewer only sees the answers of their own seat. The consolidator works in the additional consolidation seat.

For larger teams, a project can enable a strict mode. The project then lists persons and assigns papers to them. On open, each person picks their name and sees only the assigned papers in a fixed seat. Consolidation starts only when all persons have finished their papers, unless a manager overrides this rule. At the next save, SAILOR locks the finished work, and only a manager can reopen it. The strict mode is a workflow aid and not a security control. A person who edits the files directly or picks another name can still change any answer, and SAILOR states this in the dialog and in the documentation.

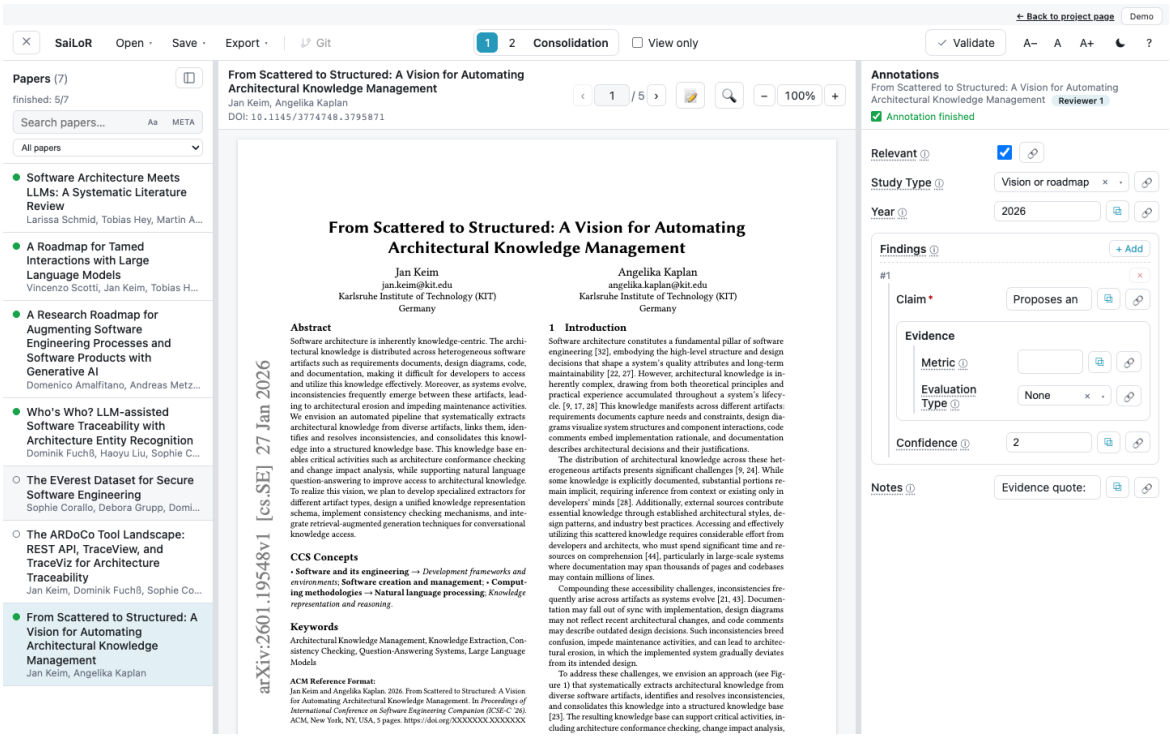



Fig. 1. Main view of SAILOR in the online demo with the paper list on the left, the PDF in the middle, and the extraction form for reviewer seat 1 on the right

Before SAILOR can compare answers, it must find out which entries of a repeatable group belong together. For example, two reviewers can list the findings of a paper in different orders. SAILOR matches the entries with a maximum-weight assignment over their pairwise similarity, and it does not match entries whose similarity is 0.5 or lower. The alignment is deterministic, so that all reviewers see the same result. Before SAILOR compares two answers, it trims them, converts them to lowercase, and collapses whitespace. If at least two reviewers answered a field and all answers are equal, SAILOR fills the consolidated value automatically and marks it as autofilled.

For the remaining fields, the consolidator opens a comparison, as shown in Figure 4. The comparison lists the answers of all reviewers. The consolidator adopts one answer, enters a different value, or marks the answers as equivalent. The disagreements of a paper or of the whole project can be exported as text, for example, to discuss them in a meeting.

SAILOR also computes the inter-rater agreement. For exactly two reviewers, it computes Cohen's $\kappa$ [31], and for two or more reviewers Fleiss' $\kappa$ [32]. In both cases, it also computes Krippendorff's $\alpha$ for nominal data [33], which tolerates missing answers. SAILOR shows the agreement overall and per field. If a statistic is not defined, for example, because all reviewers gave the same answer to every item, SAILOR reports the case instead of a number. In a screening project, the agreement only considers the decision.

### E. Collaboration with Git

SAILOR has a built-in git view that uses the git installation of the user. Instead of a line diff of JSON files, the view lists each changed field with its previous and current value. Reviewers select which field changes they commit. A pull fetches the upstream branch and reports whether the project is up to date, can be fast-forwarded, or needs a merge. SAILOR only pushes when the reviewer asks for it. It also supports branches, a project-scoped history, and stashes of the project's own files.

For a merge, SAILOR does not use the line-based merge of git. It runs a three-way merge on the level of fields. If only one side changed a field, the merge keeps that change. The schema is merged node by node, and answers under a removed field are kept. Only if both sides changed the same field to different values does the reviewer have to decide. Figure 5 shows the dialog for these conflicts. If a merge leaves conflicts in files that do not belong to the project, SAILOR aborts it and restores the previous state. Before a pull or merge starts, SAILOR shows a summary of the changed files, the changed papers, and the expected conflicts, and it waits for the reviewer to confirm. It also saves the current state of the branch, so that the reviewer can undo the last merge.

### F. Exploring the Review as a Knowledge Graph

SAILOR can turn a project into a knowledge graph. The nodes are the papers, authors, venues, keywords, and the answers of the reviewers. The edges state that a paper has an author, appears in a venue, has a keyword, cites another

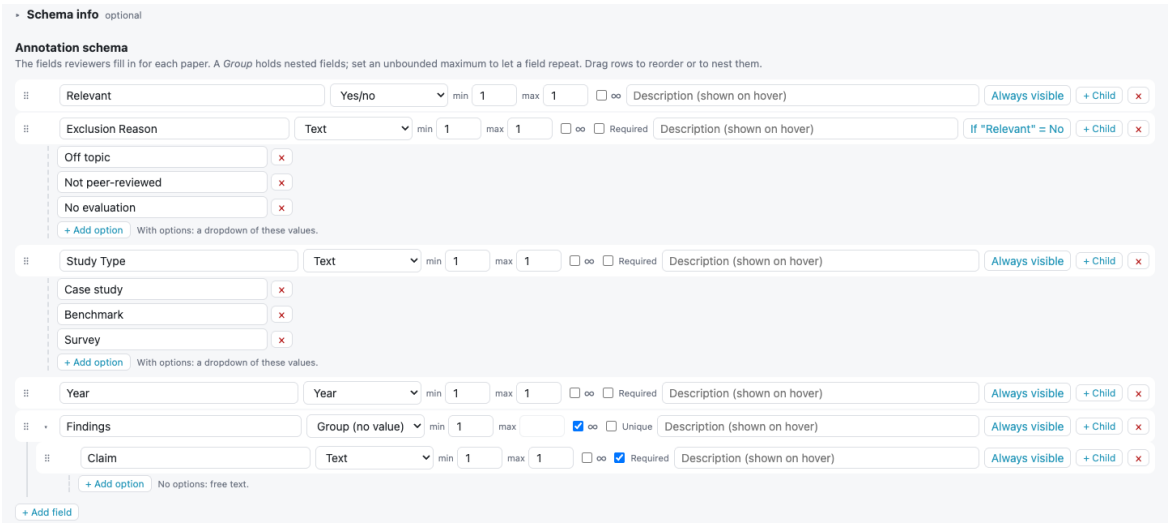



Fig. 2. Schema editor of the online demo with field types, counts, options, and a visibility condition

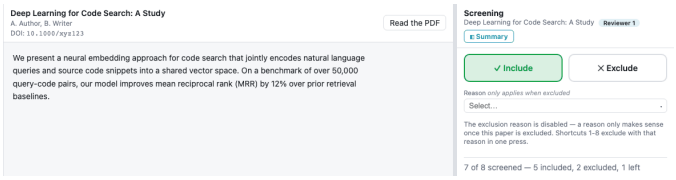



Fig. 3. Screening view of the running example in version 1.x with the abstract, the decision buttons, and the progress line

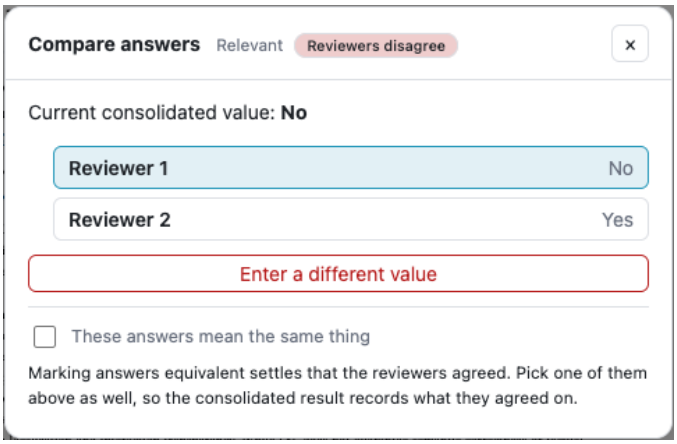



Fig. 4. Comparison of two reviewers' answers to the field *Relevant* in the consolidation seat of the online demo

paper, or has an answer. Reviewers can export the graph as JSON-LD, Resource Description Framework (RDF) Turtle, GraphML, or two comma-separated values (CSV) files. The export is deterministic, so two exports of the same project give the same files.

The knowledge graph view draws the graph in a force layout. Reviewers pan, zoom, move nodes, and expand the neighbors of a node with a double click. Each relation type has its own line style, and a legend lets reviewers hide relation

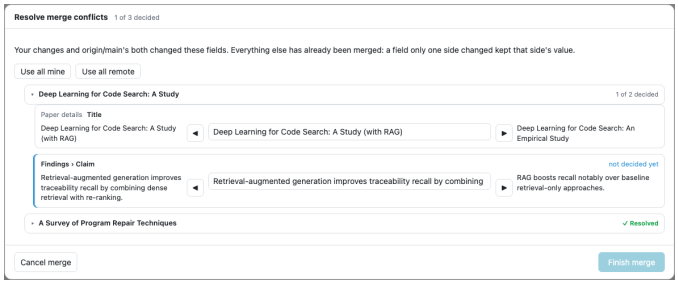



Fig. 5. Field-level resolution of merge conflicts after a pull in version 1.x

types. A "Nodes" button opens a drawer that lists the shown nodes as buttons, so that the graph is also usable without the canvas. The list shows 200 nodes at first and more on request. With several reviewers, a seat selector chooses whose answers the graph shows, including the saved consolidation. A download button in the graph area saves the graph as an SVG or PNG image, either of the visible section or of the whole graph. The image respects the current filter. A short text query filters the graph, for example, by node type or by the answer of a field.

For other questions, reviewers can write a query in a read-only subset of Cypher [34]. For example, the query `MATCH (p:Paper)-[:cites]->(q:Paper) RETURN q.year, count(*)` counts the cited papers per year. SAILOR shows the result as a table and names the position of an error in the query. Figure 6 shows a query for the papers of one author in the online demo, with the drawer open. Section V-F describes how SAILOR keeps the view and the queries fast for large graphs.

### G. Languages

The user interface is available in nine languages: English, German, Chinese, Persian, Spanish, French, Italian, Portuguese, and Japanese. A change of the language applies

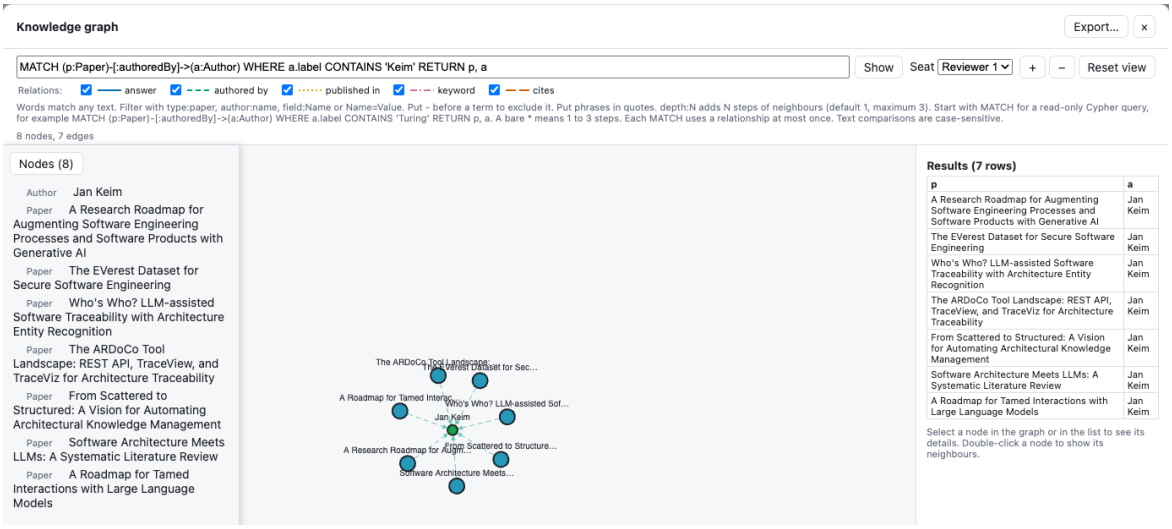



Fig. 6. Knowledge graph view of the online demo with a Cypher query, the drawer of the shown nodes on the left, and the result table on the right

at once, also to the native menus, and does not change the project. For Persian, SAILOR mirrors the layout from right to left, while code, keys, and the PDF pages stay left to right.

### H. LLM Assistance

LLM assistance is off unless the project enables it. Until then, SAILOR shows no LLM button in the toolbar and no LLM section in the settings. Reviewers can connect nine named provider application programming interfaces (APIs), among them Anthropic, OpenAI, and Google Gemini, as well as any OpenAI-compatible server. Local models run, for example, in Ollama or a llama.cpp server [35]. SAILOR stores API keys in the keychain of the operating system. Every LLM window shows a notice that the feature is a preview and that each proposal must be checked.

For extraction, the model reads the paper and proposes answers. SAILOR validates every proposal against the schema. A review table shows each proposal with the field, the value, a quote from the paper, and a confidence. In agent mode, the model can search and read pages of the paper, query OpenAlex and Crossref, and fetch web pages. SAILOR checks whether the quoted evidence occurs in the paper, and an optional judge model checks the proposals in a bounded loop. Nothing changes in the project until the reviewer applies the selected rows. Applying is one undo step and does not overwrite existing answers. SAILOR marks each field that the model wrote until the reviewer interacts with the field. For each apply, SAILOR records the provider, the model, and the time in the project, and the paper shows a disclosure line such as "AI-assisted (2×)".

In a project with several reviewers, the last seat can be dedicated to the model. The answers of this seat go through the same consolidation as the answers of the human reviewers, so that the agreement statistics also cover the model. After consolidation, an evaluation dialog compares the answers of this seat with the consolidated answers and shows the agreement per field. Reviewers can export this table as a CSV file. The project can also store its own texts for the prompts, for example, a role text or help texts per field. For screening, the model proposes a decision and a reason per paper. SAILOR never pre-selects an exclusion, and it pre-selects an inclusion only if the confidence is at least 0.8.

## V. Architecture Decisions

In this section, we describe the main architecture decisions behind SAILOR. For each decision, we state the context, the decision, and its consequences.

### A. Overview

SAILOR is a desktop application built with Tauri 2 [36]. The user interface is a React application in TypeScript that runs in the web view of the operating system. It renders PDFs with pdf.js and keeps the project state in a single store. A backend in Rust provides the native capabilities, such as file access, git, the keychain, and network requests. The user interface reaches the backend only through commands that the backend registers. At the time of writing, the TypeScript code base has about 58,400 lines of code without tests and about 55,700 lines of tests. The Rust backend has about 14,400 lines in 18 source files.

### B. Local Files Instead of a Server

Hosted tools make collaboration easy, but they keep the review data on a server (cf. Section II-B). For a research tool, this setup has drawbacks. The data can be hard to export completely, the service can change or end, and reviewers might not be allowed to upload papers. Therefore, we decided to keep all data in local files, following the idea of local-first software [29]. The project file and the annotation files are JSON files that people can read. SAILOR keeps unknown keys in the project file, so that an older version does not delete data of a newer version. Each annotation file names the schema

version it was written under, and validation reports files with an unknown schema version. The serialization is deterministic, and opening a project writes nothing. As a consequence, a version control system only sees changes that a reviewer made. The price of this decision is that SAILOR needs its own collaboration mechanism.

### C. Git as Collaboration Layer

We decided to use git for collaboration instead of building a synchronization server. Git keeps a full history of changes [30], and reviewers can host a repository where their institution allows it. To keep merges small, SAILOR splits the annotations into one file per paper and seat. Two reviewers who work on different seats never edit the same annotation file.

However, a line-based merge does not understand JSON. When two people edit the same file, for example, the consolidator and a reviewer who changes the schema, git can report conflicts in lines that do not conflict in their meaning. Therefore, SAILOR runs its own three-way merge on the parsed files. It compares both sides to the common base field by field. A side that did not change a value does not influence the result. Repeatable entries, project settings, and schema nodes are merged separately, and answers under a removed schema field are kept.

Using the git installation of the user brings risks. A cloned repository can contain a configuration that runs code, for example, hooks or a file system monitor. SAILOR runs git only in the backend and overrides these settings on every call, including the pager, the editor, and the `ext` protocol. It disables interactive prompts, limits the run time of each command, and validates clone URLs, paths, and branch names. Git operations are only allowed on repositories that the user opened or cloned in the current session. Before SAILOR commits or discards selected field changes, it checks that the file on disk still matches the reviewed state, and it refuses the write otherwise.

### D. From Electron to Tauri

Up to version 1.x, SAILOR was an Electron application [37]. The Electron main process had about 5,600 lines of TypeScript and 98 inter-process communication (IPC) handlers. Electron bundles its own Chromium and Node.js runtime, and the main process could use the full Node.js API.

For version 2.0.0, we moved SAILOR to Tauri 2 [36]. Tauri uses the web view of the operating system and a backend in Rust. In Tauri, the security boundary lies in the Rust backend. Tauri also offers a capability system and a signed updater. We split the responsibilities as follows. Rust owns everything that needs authorization: file input and output, process spawning, downloads, and access to the keychain. TypeScript keeps the parsing of files, the merge logic, and the assembly of exported PDFs, because this code was already tested and does not need native access. SAILOR does not expose the generic file system and shell plugins of Tauri to the user interface. The capability file grants the web view only the default core permissions and the permission to set the window title. An application manifest lists the commands of the backend, so that the web view can only call these commands.

The move has consequences for users and for us. SAILOR now depends on three different web views: WKWebView on macOS, WebView2 on Windows, and WebKitGTK on Linux. The continuous integration (CI) probe renders a PDF in all three web views. The minimum macOS version is 15. Users of version 1.x have to install version 2.0.0 manually and enter their API keys again. The guard against reload shortcuts is ported, but we have not tried it by hand on Windows and Linux (cf. Section VII).

### E. Security Model

A review tool opens files and repositories from other people, and it can send data to LLM providers. Therefore, we treat the user interface as untrusted and enforce all checks in the backend. SAILOR only saves a project to a path that the user opened or chose in a native dialog during the current session. It serves a PDF only if the file lies inside the project folder. For a PDF outside the folder, SAILOR asks the user and allows only this exact path for the current session. Exports only write to destinations that the user picked in a save dialog, and SAILOR refuses symbolic links as targets. Updates are signed, and SAILOR downloads and installs them only when the user asks for it.

We added further safeguards for version 2.0.0. A content security policy (CSP) lets the web view run scripts only from the application itself. The backend writes project, annotation, and PDF files to a temporary file first and then renames it, so that a crash does not leave a half-written file. It limits the responses of LLM providers and the output of git to 32 MiB each.

Before the release of version 2.0.0, we reviewed the code for security and data loss and fixed the findings. SAILOR now sets the project folder for PDF access only for a project file that the user opened or chose in the current session. It refuses to overwrite a PDF outside the project folder. Before the first write, it checks every target of a split project and every annotation file that it deletes for symbolic links and for paths that leave the project folder. A save that is still running when the user opens another project stays on its own project, and a waiting save does not write while the new project loads. If git rewrites the project files while the project has unsaved edits, SAILOR keeps the edits and does not load the rewritten files.

For LLM providers, SAILOR stores API keys in the keychain and sends a key only to the origin of its provider. It refuses redirects for requests that carry a key. It also refuses to send a key over plain `http` to a host outside the local network. The agent mode can fetch web pages, which opens the door to server-side request forgery (SSRF) [38]. SAILOR only allows `http` and `https` URLs, resolves the host name, and refuses requests to loopback, private, and other non-public addresses. It checks every redirect again, follows at most five redirects, stops after 20 seconds, reads at most

2 MB, and only accepts text content. These requests ignore the proxy of the system, because a proxy resolves the host name itself and would bypass the address check. A model that follows injected instructions could also try to send data out in a URL. Therefore, the tool refuses URLs with more than 2,048 characters and removes the query string and the fragment before the request. The fetched text and the results of the literature search and the DOI lookup are marked as untrusted data. The system prompt tells the model to never follow instructions from such data.

### F. Large Projects

A mapping study can start with tens of thousands of candidate papers. For such projects, opening the project, drawing the knowledge graph, and running a query can block the user interface. Therefore, we moved this work out of the critical path.

SAILOR opens a project in the split layout in two steps. First, the Rust backend reads only the project file, and the user interface shows the metadata of all papers. Then, the user interface loads the annotation files in batches of 250 papers and gives control back to the event loop between two batches. A progress line shows how many papers are loaded, and reviewers can already annotate the loaded papers. Until all papers are loaded, SAILOR blocks every save and disables the features that need the whole project, for example, validation, the git view, and the consolidation overview. The backend reads a batch only for the opened project and only for the papers that the project file listed when the user opened it. If a batch fails, the project stays open, and the user can retry the load. A project in the older single-file layout still opens in one step, because the backend must parse the file as a whole. The backend also parses the project file only once per open and reads each batch of annotation files in one pass. We profiled a project with 50,000 papers once on a laptop under load. In this profile, the time in the Rust backend to open the project and to read all batches dropped from about 6.6 s to about 2.0 s. This time excludes the rendering in the user interface.

The knowledge graph view has no limit on the number of nodes. It draws the graph on a canvas instead of one page element per node, and it skips nodes outside the visible area. It draws labels only when the user zooms in and at most 2,000 nodes are visible. A web worker computes the force layout, sends the positions to the view, and stops when the layout is stable. For more than 1,000 nodes, the worker uses fewer steps and a coarser approximation. Benchmark tests with a generated graph of 50,000 nodes require that the view prepares a frame in less than 1 s and finds the node under the pointer in less than 1 ms on average.

Queries in Cypher run in a second web worker. The worker receives a copy of the graph once and builds an index for it. A new query cancels the running query, and a query stops after 20 s. We implemented only a read-only subset of Cypher. The parser refuses all clauses that write, such as `CREATE`, `SET`, and `DELETE`, and `count(*)` is the only function. A query can have at most 4,000 characters, and a pattern can have at most three hops. A result has at most 10,000 rows. After 2,000,000 evaluation steps, SAILOR stops the query and marks the result as incomplete. These limits keep a query from blocking the tool, for example, a query that joins every node with every other node. On a generated graph with 150,000 nodes and 600,000 relationships, the tests require less than 1 s for a filter by label, 3 s for a join over one relationship, and 5 s for a count per year. In one run on a laptop under load, the copy and the index took about 1.0 s, and the three queries took about 15 ms, 79 ms, and 123 ms.

### G. Controlled LLM Assistance

Prior studies found that LLMs can help with screening and extraction but still make errors [5], [6]. Therefore, we designed the LLM support so that it cannot change the review data without a human decision. Suggestions live outside the project until the reviewer applies them. SAILOR validates them against the schema and checks quoted evidence against the text of the paper. The durable record of each apply makes the use of LLMs visible in the data, so that authors can report it. Treating the model as an additional reviewer seat reuses the consolidation and agreement statistics, instead of adding a separate evaluation mechanism.

Some reviewers cannot send papers to a hosted provider. For them, SAILOR supports local models. For chat models, users connect a local server such as Ollama, LM Studio, llama.cpp, or vLLM. For small decision models, SAILOR can manage a llama.cpp server itself [35]. It downloads the server and the models only from an allowlist of sources, verifies their SHA-256 checksums, and starts the server on `127.0.0.1` with a random key per session. The server uses a graphics processing unit (GPU) when one is available and the central processing unit (CPU) otherwise. We verified this runtime on macOS on arm64. The managed installation needs a llama.cpp release with a specific route for decision models. As long as no release includes this route, the setup dialog keeps the installation disabled and points users to a server that they run themselves.

## VI. Requirements and Traceability

SAILOR asks reviewers to keep their decisions traceable. We think that the tool should meet the same standard for its own behavior. Traceability links requirements to the artifacts that realize and verify them [39], [40]. In this section, we describe how we documented the requirements of SAILOR and how far they are traced. We also describe the tests that verify them.

### A. Recovery of the Requirements

SAILOR grew without a written specification. Therefore, we reverse-engineered the requirements from the implementation of version 1.8.1 in August 2026. We kept them up to date until November 2026 as the code changed for version 2.0.0. Every requirement describes behavior that the code implements, and every link is a recovered link and not one that we created during development.

TABLE II
REQUIREMENTS PER CATEGORY AND TRACE LINKS OF ALL 510 REQUIREMENTS

| Category | Requirements | |
|---|---|---|
| Data model and project editor | 92 | |
| Annotation view, paper list, and PDF | 76 | |
| Screening | 39 | |
| Consolidation | 61 | |
| Git integration | 67 | |
| LLM assistance | 104 | |
| Platform and desktop shell | 71 | |
| **Trace link** | **Count** | **Share** |
| Implementing source code | 484 | 95% |
| At least one verifying test | 480 | 94% |
| Source inspection only | 30 | 6% |
| Introducing or pinning commit | 122 | 24% |
| Documentation section | 410 | 80% |

We used four recovery techniques, in descending order of evidential weight. First, we read the names and assertions of tests as executable specifications. For example, a store test named "clears the reason in the same undo step" pins the requirement that a paper loses its exclusion reason in the same undo step when its decision changes from "exclude" to another state. Second, we inspected the source code and recorded the implementing files and lines. Third, we searched the commit history for the commits that introduced or pinned a behavior. Fourth, we mapped each requirement to the developer wiki, the schema reference, and the user guide.

Each requirement has an identifier with a category prefix, for example, `REQ-CON-80` for consolidation. The identifiers advance in steps of ten, so that new requirements fit in without renumbering. An entry consists of a title, a description in the form "The system shall …", a type, the evidence, a status, and the tests. The type states whether the requirement is functional or non-functional and names its quality characteristic according to ISO/IEC 25010 [41]. The requirements are split into seven files by category. A separate traceability matrix lists one row per requirement with the columns *Implemented in*, *Verified by*, *History*, and *Docs*. A dash in the *Verified by* column means that we confirmed the requirement only by inspecting the source code.

### B. Coverage

In total, we documented 510 requirements. Table II shows how they are distributed over the categories and how many of them are linked to each kind of artifact. Of the requirements, 448 are functional and 62 are non-functional, and 46 have the type security. At the time of writing, all 510 requirements are implemented. For three of them, the evidence is weaker than for the rest. We have not tried the guard against reload shortcuts by hand on Windows and Linux. We checked the right-to-left layout for Persian only in the simulated browser of the unit tests. We checked the container image with a remote desktop in the browser only by hand. The links point to 230 distinct source files and 238 distinct test files. Of all requirements, 95% link to implementing source code, and 94% link to at least one verifying test. The remaining 30 requirements are confirmed by source inspection only. Of these, 15 belong to the platform and desktop shell and 7 to the git integration. Only 24% link to a commit that introduced or pinned the behavior.

We count these numbers with a small script in the report folder that parses the requirement files and the matrix. The script also checks that every linked file exists. At the time of writing, four new requirements from the review before the release (cf. Section V-E) have no row in the matrix yet. For them, the script takes the links from the requirement files.

Since November 2026, the CI also checks the trace links with OpenFastTrace [42]. A script converts the requirement files into the input format of OpenFastTrace. Source files and tests carry tags that name the requirement that they implement or verify. The CI job fails if a requirement that names source code or tests lacks a matching tag.

To keep the links current, the repository asks every change that affects a requirement to update the matching requirement file in the same change. OpenFastTrace checks the tags in the code, but no check compares the matrix with the requirement files. Thus, the matrix can still drift, as the four missing rows show.

### C. Automated Tests

SAILOR has four test suites. The unit suite has 3709 tests in 251 files and runs with Vitest in jsdom, a simulated browser environment. The integration suite has 56 tests in 11 files. These tests render the real components and run scenarios such as annotation, consolidation, and merges against scratch git repositories. The Rust backend has 135 tests. Finally, a probe build of the desktop application opens the sample project, renders a PDF, and checks that a path traversal is refused. The CI runs this probe in the system web view on six runners: two versions of macOS, Windows, two versions of Ubuntu, and Ubuntu on arm64. On Linux and Windows, it also drives the application through WebDriver.

Every CI job runs a script from the repository, so that developers can run the same checks locally. The build job fails if the line coverage of the unit tests drops below 82%. At the time of writing, the unit tests cover 84.6% of the lines and 88.2% of the branches. Further jobs lint the Rust code, the workflows, and the shell scripts, and they audit the npm and Rust dependencies for known vulnerabilities. The CI pins every action to a commit and runs with a read-only token. The release workflow runs in a separate environment, and it publishes software bill of materialss (SBOMs), checksums, and build attestations with the installers.

### D. Mutation Testing

Coverage shows which code the tests run, but not whether the tests would notice a wrong result. Mutation testing measures this property [43]. It changes the code in small ways and checks whether a test fails. We used StrykerJS [44] on the model code and on the pure modules of the LLM assistance.

A full run over the model code took too long, so we stopped it at about 85% of the mutants. Instead, we ran StrykerJS file by file. For each surviving mutant, we either added a test that kills it or recorded why no test can kill it.

We finished this process for 35 files. In these files, the mutation score rose from between 54% and 100% to between 80% and 100%. For example, the score of the code that reads and writes projects rose from 84.5% to 96.8%. We found no bug in the production code. All remaining survivors that we inspected are equivalent mutants or lie in code that no input can reach. The work added about 350 unit tests, which the count of 3709 unit tests above includes. Mutation testing is not part of the CI, because a full run takes more than one hour. Developers can start it in a manual workflow.

## VII. Discussion and Limitations

In this section, we discuss the limitations of SAILOR and of this report. For each limitation, we state what it affects and how we mitigate it.

### A. Limitations

We have not evaluated SAILOR with users. This report describes the tool and its design, but it does not show that SAILOR makes reviews faster, more accurate, or easier to trace than other tools. We derived the design from the guidelines for SLRs, and it might not match the needs of every research group. A user study is needed to assess the benefit (cf. Section VIII).

The requirements were reverse-engineered from the code by the developer of the tool. Thus, they describe what SAILOR does and not what users need. If the code has a bug, a requirement can describe the buggy behavior as intended. Also, the developer might have overlooked behavior or described it in favor of the implementation. We mitigate this threat with the evidence of each requirement, because tests, source lines, and commits can be checked by others. Still, an independent review of the requirements is open.

The move to Tauri is recent. The guard against reload shortcuts is now ported to Tauri as well. However, no automated test shows its dialog in a running application, and we have not tried it by hand on Windows and Linux. Furthermore, Figure 3 and Figure 5 still show the user interface of version 1.x, because the online demo has no screening project and no git view.

Our tests cover the three operating systems unevenly. The probe of the desktop application runs on macOS, Windows, and Linux, but a WebDriver session drives it only on Linux and Windows, because the Tauri driver does not support macOS. Most behavior is tested in the unit and integration suites, which run in a simulated browser and not in the web views of the operating systems. Differences between WKWebView, WebView2, and WebKitGTK can therefore remain unnoticed. The CI probe renders a PDF in all three web views, but the probe does not yet cover further workflows. Likewise, we checked the right-to-left layout for Persian only in the simulated browser.

The performance numbers in this report come from benchmark tests with generated graphs and from single runs on a laptop under load. They show that the limits work, but they do not predict the speed on the computer of a reviewer. A real review graph can have a different structure than the generated graphs.

The mutation testing covers only the model code and two modules of the LLM assistance. The user interface components, the store, and the Rust backend are not mutated. Also, we never completed a full run over the model code, only runs per file.

The strict mode helps a team to follow its process, but it does not enforce the process. A person with access to the files can change any answer. Teams that need this protection have to rely on the access rights of their git hosting.

The installers are not signed with a certificate from Apple or Microsoft. macOS and Windows warn users the first time they open SAILOR, and users have to confirm the start. This step can confuse users and can also train them to ignore such warnings. The README explains the steps for each operating system. Updates are signed with a key of the project, and SAILOR verifies the signature before it installs an update.

We verified the managed local runtime for decision models only on macOS on arm64. The code paths for GPUs with CUDA and Vulkan have not run on real hardware. In addition, the managed installation stays disabled until llama.cpp releases the route that the decision models need. Until then, users who want local decision models have to run a server themselves. Hosted providers and local chat servers are not affected by this limitation.

The LLM assistance is a preview. We have not measured how accurate its proposals are, and studies report errors of LLMs in screening and extraction [5], [6]. A reviewer who accepts proposals without checking them can bring wrong data into the review. SAILOR reduces this risk. It checks the quoted evidence, marks the fields that the model wrote, and records each use for the report. It cannot replace the judgment of the reviewer.

SAILOR does not cover all activities of an SLR. It records the search strings and databases, but it does not run searches. It also does not synthesize the extracted data. The knowledge graph view and its queries help to explore the data, but reviewers still need their own scripts or tools for the analysis.

### B. Availability

SAILOR is open source under the GPL-3.0 license. The source code is available on GitHub.[1] The release workflow builds installers for macOS 15 or later on Apple silicon and Intel, for Windows, and for Linux on x86-64 and arm64 as AppImage, Debian, and RPM packages. The Zenodo concept DOI [14] resolves to the latest archived version. A project site on GitHub Pages hosts the online demo.[2] The demo runs in the browser without an installation and saves nothing. A second

[1] https://github.com/Gram21/SaiLoR
[2] https://gram21.github.io/SaiLoR/

mode of the demo opens the schema editor and downloads the project file on save. The repository also contains a sample screening project and a sample extraction project, a user guide, and the requirement files described in Section VI.

## VIII. Conclusion

In this paper, we looked into tool support that keeps the data of a systematic literature review (SLR) traceable. We presented SAILOR, an open-source desktop tool for screening, data extraction, and consolidation. A SAILOR project consisted of plain JavaScript Object Notation (JSON) files that a schema described and that git versioned. Reviewers worked independently in their own seats and linked their answers to evidence in the papers. A consolidator resolved disagreements with the help of agreement statistics, and SAILOR merged concurrent changes field by field. Optional large language model (LLM) assistance proposed answers with evidence, but a human reviewed every proposal before SAILOR applied it.

We also explained the main architecture decisions. These were the local files instead of a server, git as collaboration layer, the security model, the move from Electron to Tauri in version 2.0.0, and the support for large projects. To keep the tool itself traceable, we documented 510 requirements. Of these requirements, 95% linked to implementing source code and 94% linked to at least one automated test. The test suites held 3709 unit tests, 56 integration tests, and 135 Rust tests. Mutation testing of the model code added about 350 unit tests and found no bug in the production code. We made the source code of SAILOR available on GitHub under the GPL-3.0 license, with a Zenodo concept digital object identifier (DOI) for all versions. An online demo let readers try SAILOR in the browser without an installation.

Overall, we argued that plain files and git can keep review data traceable, but the benefit for reviewers remains open.

Regarding the tool, we want to retest the user interface, including the guard against reload shortcuts, in the web views of all three operating systems. We also want to extend the mutation testing to the store, the user interface components, and the Rust backend. In addition, we want to verify the local model runtime on graphics processing units (GPUs) with CUDA and Vulkan and enable the managed installation once llama.cpp provides the required route.

In future research, we want to evaluate SAILOR in a user study with reviewers who conduct a real SLR. We want to find out whether the tool reduces the effort of extraction and consolidation and whether the recorded history helps others to trace the review. We also want to measure how accurate the LLM proposals are and how they influence the decisions of reviewers.

Lastly, we want to have the requirements reviewed by people other than the developer and check the traceability matrix automatically in the continuous integration (CI).

## References


[1] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Keele University and Durham University, Tech. Rep. EBSE-2007-01, 2007.

[2] K. Petersen, S. Vakkalanka, and L. Kuzniarz, "Guidelines for conducting systematic mapping studies in software engineering: An update," *Information and Software Technology*, vol. 64, pp. 1–18, 2015.

[3] C. Wohlin, "Guidelines for snowballing in systematic literature studies and a replication in software engineering," in *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering (EASE)*. ACM, 2014, pp. 1–10.

[4] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, R. Chou, J. Glanville, J. M. Grimshaw, A. Hróbjartsson, M. M. Lalu, T. Li, E. W. Loder, E. Mayo-Wilson, S. McDonald, L. A. McGuinness, L. A. Stewart, J. Thomas, A. C. Tricco, V. A. Welch, P. Whiting, and D. Moher, "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews," *BMJ*, vol. 372, p. n71, 2021.

[5] K. R. Felizardo, M. S. Lima, A. Deizepe, T. U. Conte, and I. Steinmacher, "ChatGPT application in systematic literature reviews in software engineering: an evaluation of its accuracy to support the selection activity," in *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. ACM, 2024, pp. 25–36.

[6] Q. Khraisha, S. Put, J. Kappenberg, A. Warraitch, and K. Hadfield, "Can large language models replace humans in systematic reviews? Evaluating GPT-4's efficacy in screening and extracting data from peer-reviewed and grey literature in multiple languages," *Research Synthesis Methods*, vol. 15, no. 4, pp. 616–626, 2024.

[7] C. Marshall and P. Brereton, "Tools to support systematic literature reviews in software engineering: A mapping study," in *ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, 2013, pp. 296–299.

[8] C. Marshall, P. Brereton, and B. Kitchenham, "Tools to support systematic reviews in software engineering: a feature analysis," in *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering (EASE)*. ACM, 2014, pp. 1–10.

[9] H. Harrison, S. J. Griffin, I. Kuhn, and J. A. Usher-Smith, "Software tools to support title and abstract screening for systematic reviews in healthcare: an evaluation," *BMC Medical Research Methodology*, vol. 20, p. 7, 2020.

[10] Veritas Health Innovation, "Covidence systematic review software," Melbourne, Australia. https://www.covidence.org, accessed 2026-11-01.

[11] M. Ouzzani, H. Hammady, Z. Fedorowicz, and A. Elmagarmid, "Rayyan—a web and mobile app for systematic reviews," *Systematic Reviews*, vol. 5, no. 1, p. 210, 2016.

[12] C. Kohl, E. J. McIntosh, S. Unger, N. R. Haddaway, S. Kecke, J. Schiemann, and R. Wilhelm, "Online tools supporting the conduct and reporting of systematic reviews and systematic maps: a case study on CADIMA and review of existing tools," *Environmental Evidence*, vol. 7, no. 1, p. 8, 2018.

[13] R. van de Schoot, J. de Bruin, R. Schram, P. Zahedi, J. de Boer, F. Weijdema, B. Kramer, M. Huijts, M. Hoogerwerf, G. Ferdinands, A. Harkema, J. Willemsen, Y. Ma, Q. Fang, S. Hindriks, L. Tummers, and D. L. Oberski, "An open source machine learning framework for efficient and transparent systematic reviews," *Nature Machine Intelligence*, vol. 3, no. 2, pp. 125–133, 2021.

[14] J. Keim, "SaiLoR 2.0.0," Software. Source code: https://github.com/Gram21/SaiLoR, 2026, zenodo concept DOI, always resolves to the latest version.

[15] B. A. Kitchenham, D. Budgen, and P. Brereton, *Evidence-Based Software Engineering and Systematic Reviews*. Chapman and Hall/CRC, 2015.

[16] E. Hassler, J. C. Carver, N. A. Kraft, and D. Hale, "Outcomes of a community workshop to identify and rank barriers to the systematic literature review process," in *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering (EASE)*. ACM, 2014, pp. 1–10.

[17] A. Al-Zubidy and J. C. Carver, "Identification and prioritization of SLR search tool requirements: an SLR and a survey," *Empirical Software Engineering*, vol. 24, no. 1, pp. 139–169, 2019.

[18] K. R. Felizardo and J. C. Carver, "Automating systematic literature review," in *Contemporary Empirical Methods in Software Engineering*, M. Felderer and G. H. Travassos, Eds. Springer, 2020, pp. 327–355.

[19] R. van Dinter, B. Tekinerdogan, and C. Catal, "Automation of systematic literature reviews: A systematic literature review," *Information and Software Technology*, vol. 136, p. 106589, 2021.

[20] V. Freitas, "Parsifal," Software. https://parsif.al, source code at https://github.com/vitorfs/parsifal, accessed 2026-11-02.

[21] E. Hernandes, A. Zamboni, S. Fabbri, and A. Di Thommazo, "Using GQM and TAM to evaluate StArt – a tool that supports systematic review," *CLEI Electronic Journal*, vol. 15, no. 1, 2012.

[22] S. Fabbri, C. Silva, E. Hernandes, F. Octaviano, A. Di Thommazo, and A. Belgamo, "Improvements in the StArt tool to better support the systematic review process," in *Proceedings of the 20th International Conference on Evaluation and Assessment in Software Engineering (EASE)*. ACM, 2016, pp. 1–5.

[23] A. M. Fernández-Sáez, M. Genero Bocco, and F. P. Romero, "SLR-TOOL - a tool for performing systematic literature reviews," in *Proceedings of the 5th International Conference on Software and Data Technologies (ICSOFT)*. SciTePress, 2010, pp. 157–166.

[24] E. Syriani, I. David, and G. Kumar, "Screening articles for systematic reviews with ChatGPT," *Journal of Computer Languages*, vol. 80, p. 101287, 2024.

[25] K. R. Felizardo, I. Steinmacher, M. S. Lima, A. Deizepe, T. U. Conte, and M. P. Barcellos, "Data extraction for systematic mapping study using a large language model - a proof-of-concept study in software engineering," in *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. ACM, 2024, pp. 407–413.

[26] G. Gartlehner, L. Kahwati, R. Hilscher, I. Thomas, S. Kugley, K. Crotty, M. Viswanathan, B. Nussbaumer-Streit, G. Booth, N. Erskine, A. Konet, and R. Chew, "Data extraction for evidence synthesis using a large language model: A proof-of-concept study," *Research Synthesis Methods*, vol. 15, no. 4, pp. 576–589, 2024.

[27] S. Wang, H. Scells, B. Koopman, and G. Zuccon, "Can ChatGPT write a good Boolean query for systematic review literature search?" in *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2023, pp. 1426–1436.

[28] A. Alshami, M. Elsayed, E. Ali, A. E. E. Eltoukhy, and T. Zayed, "Harnessing the power of ChatGPT for automating systematic review process: Methodology, case study, limitations, and future directions," *Systems*, vol. 11, no. 7, p. 351, 2023.

[29] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: you own your data, in spite of the cloud," in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*. ACM, 2019, pp. 154–178.

[30] K. Ram, "Git can facilitate greater reproducibility and increased transparency in science," *Source Code for Biology and Medicine*, vol. 8, no. 1, p. 7, 2013.

[31] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[32] J. L. Fleiss, "Measuring nominal scale agreement among many raters," *Psychological Bulletin*, vol. 76, no. 5, pp. 378–382, 1971.

[33] K. Krippendorff, *Content Analysis: An Introduction to Its Methodology*, 4th ed. SAGE Publications, 2019.

[34] N. Francis, A. Green, P. Guagliardo, L. Libkin, T. Lindaaker, V. Marsault, S. Plantikow, M. Rydberg, P. Selmer, and A. Taylor, "Cypher: An evolving query language for property graphs," in *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*. ACM, 2018, pp. 1433–1445.

[35] ggml-org, "llama.cpp: LLM inference in C/C++," https://github.com/ggml-org/llama.cpp, accessed 2026-11-02.

[36] Tauri Contributors, "Tauri 2.0," https://v2.tauri.app/, accessed 2026-11-02.

[37] OpenJS Foundation, "Electron: Build cross-platform desktop apps with JavaScript, HTML, and CSS," https://www.electronjs.org/, accessed 2026-11-02.

[38] OWASP Foundation, "Server side request forgery," https://community.owasp.org/attacks/Server_Side_Request_Forgery, accessed 2026-11-02.

[39] O. C. Z. Gotel and A. C. W. Finkelstein, "An analysis of the requirements traceability problem," in *Proceedings of the IEEE International Conference on Requirements Engineering (ICRE)*. IEEE, 1994, pp. 94–101.

[40] J. Cleland-Huang, O. C. Z. Gotel, J. Huffman Hayes, P. Mäder, and A. Zisman, "Software traceability: trends and future directions," in *Future of Software Engineering Proceedings (FOSE)*. ACM, 2014, pp. 55–69.

[41] ISO/IEC, "ISO/IEC 25010:2011 systems and software engineering – systems and software quality requirements and evaluation (SQuaRE) – system and software quality models," International Organization for Standardization, 2011.

[42] OpenFastTrace Contributors, "OpenFastTrace: Requirement tracing suite," https://github.com/itsallcode/openfasttrace, accessed 2026-11-06.

[43] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[44] Stryker Mutator Contributors, "StrykerJS: Mutation testing for JavaScript and TypeScript," https://stryker-mutator.io/, accessed 2026-11-06.